



SIMON

WHITEPAPER

Rozproszony habitat dla otwartych wag modeli. Ludzie z GPU liczą inferencję, sieć weryfikuje, że policzyli naprawdę.

Wersja: 1.0 · 2026-09-17

Status: architektura + rdzeń protokołu; sieć uruchomiona na dwóch maszynach (test), weryfikacja obliczeń
ZAPROJEKTOWANA, nie zaimplementowana

Autorstwo koncepcji: Karol (Szpili) · implementacja i pomiary: Szpon (OpenClaw) + Claude Code

Zasada nadrzędna: **bez dowodu nie ma statusu** (ZAPROJEKTOWANE / ZAIMPLEMENTOWANE / ZMIERZONE).

1. Streszczenie

SIMON to **rozproszony habitat dla otwartych wag modeli**. Ludzie z GPU liczą inferencję, dostają za to token, a sieć weryfikuje, że policzyli naprawdę — bez jednego serwera, który trzeba wyłączyć.

Model to dziś plik do pobrania. Plik nie ma gdzie żyć poza cudzym API. SIMON to miejsce, gdzie wagi **zamieszkują**: działają, są używane i rozwijają się, a dostęp do nich płynie z sieci, nie z jednej chmury.

Różnica wobec Hugging Face jest całą różnicą projektu:

	Hugging Face	SIMON
Model	plik do pobrania	proces, który działa
Stan	zamrożony w momencie uploadu	żyje, jest używany, ewoluuje
Kryterium bytu	„ktoś wrzucił”	używany + sprawny + bezpieczny
Wartość	liczba pobrań	realne użycie i jakość
Metafora	biblioteka	habitat

HF przechowuje. SIMON utrzymuje przy życiu.

2. Problem

Trzy zjawiska naraz:

2.1. Otwarte wagi nie mają infrastruktury. Model udostępniony publicznie stoi na jednym serwerze albo czeka na pobranie. Jeśli serwer zniknie — model znika. Nie ma mechanizmu, który dawałby mu trwałe miejsce działania.

2.2. Dostęp do modeli jest scentralizowany. Nawet gdy wagi są otwarte, praktyczny dostęp idzie przez API kilku firm. Kto ma API, ustala cenę, limity i zasady użycia danych.

2.3. Domowy sprzęt leży bezczynnie. Karty graficzne konsumentów są bezczynne przez większość doby. BOINC i Salad pokazały, że ludzie chętnie oddają moc — ale nie za darmo i bez weryfikacji pracy.

Czego brakuje: mechanizmu, który płaci za realnie wykonaną i sprawdzalną pracę na cudzym sprzęcie.

3. Teza

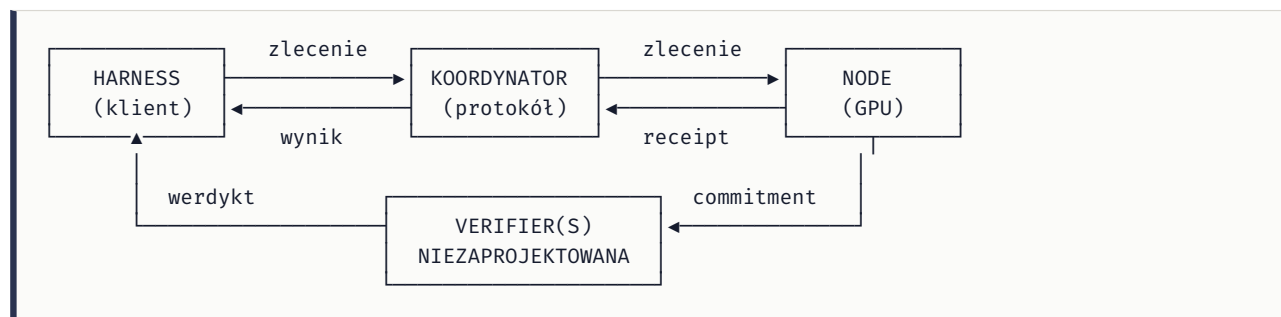
THINK za zweryfikowaną pracę; spalona opłata jako warunek emisji.

Trzy elementy, nierozdzielne:

1. **Weryfikacja** — sieć musi umieć sprawdzić, że node policzył naprawdę, a nie tylko twierdzi.
2. **Losowanie** — klient nie wybiera node'a, więc nie może kupić „swojego”.
3. **Rozliczenie** — praca płaci, oszustwo traci stawkę.

Bez weryfikacji cała reszta nie ma sensu: node, który sam podpisze fałszywy wynik, przechodzi każdą kontrolę. **To jest punkt, w którym SIMON albo stoi, albo leży.**

4. Architektura



Cztery warstwy:

Warstwa	Rola	Technologia
Klient	zlecenie pracy, odbiór wyniku	ACP (JSON-RPC), integracja z dowolnym harnessem
Koordinator	przydział, rozliczenie, rejestr	Rust
Node	liczenie modelu, składanie receiptu	Rust + vLLM (Python)
Transport	ogłoszenia, dial, wymiana	libp2p (gossipsub + request-response)

Przepływ:

```

klient rozgłasza zlecenie (gossipsub – nie wie, do kogo)
↓
koordynatorzy odpowiadają → wygrywa jeden (deterministyczny ranking)
↓
node z GPU liczy model i składa RECEIPT
↓
klient sprawdza SAM podpis i spójność receiptu (nie ufa koordynatorowi)
↓
[VERIFIER] odtwarza aktywacje i wydaje werdykt ← ROLA NIEZAPROJEKTOWANA

```

Receipt to podpisane twierdzenie, nie dowód. Node składa twierdzenie o wykonaniu wraz z materiałem do probabilistycznego audytu. Sam podpis potwierdza **autora receiptu**, nie poprawność obliczenia.

Co klient może sprawdzić sam, a czego nie. Sam weryfikuje podpis, zgodność `job_id` i modelu oraz — od 2026-09-18 — **czy odcisk w receiptcie pokrywa faktycznie odebraną treść** (wcześniej podpis obejmował tylko metadane, więc node mógł odesłać dowolny tekst; patrz `docs/SECURITY-LOG.md`).

KOREKTA. Wcześniejsza wersja tego akapitu mówiła, że klient weryfikuje „jednorazowość receiptu”. **Takiej kontroli nie ma.** Ponownemu użyciu receiptu zapobiega co innego: `order_id` jest odciskiem zlecenia ze świeżym nonce, więc `job_id` jest nieprzewidywalny i receipt z cudzego zlecenia nie przechodzi bramki. To skutek konstrukcji, nie sprawdzenie — i nie daje wykrywania powtórzeń między klientami ani w czasie. Do rozliczeń potrzebny jest rejestr (M5.2), bo bez niego nic nie zauważy policzenia tego samego receiptu dwa razy.

Nie jest w stanie sam stwierdzić, czy node faktycznie policzył — do tego potrzebne są wagi modelu i moc na odtworzenie aktywacji. Klient bez GPU albo ufa komuś trzeciemu, albo nie ma weryfikacji obliczeń wcale. Stąd osobna rola:

VERIFIER — rola WYMAGANA, ale NIEZAPROJEKTOWANA. Niezależny weryfikator odtwarza wskazane aktywacje i wydaje podpisany werdykt. Jego przydział, sposób losowania, wynagrodzenie i odporność na koluzję są **wymagane, ale niezaprojektowane**. Nie wpisujemy tu jeszcze „komitet 3-7” ani quorum — to hipotezy projektowe, nie decyzje.

Decyzja architektoniczna: SIMON jest **agentem ACP**, nie klientem. Harness widzi jednego agenta; swarm jest niewidoczny. Jeden adapter → działa z Claude Code, OpenClaw, dsh i każdym innym harnessem mówiącym ACP. (ACP to standard Zed, Apache-2.0 — niezależny od jednego dostawcy.)

5. Weryfikacja — serce projektu

Trzy warstwy:

5.1. TopLoc — probabilistyczny audyt aktywacji.

Fingerprint aktywacji wybranej warstwy daje **sygnał**, że deklarowany model przetwarzał dane wejście. Weryfikator odtwarza wskazane aktywacje i porównuje je z commitmentem wykonawcy.

Mechanizm **nie jest pełnym kryptograficznym dowodem całej inferencji**: jego gwarancje zależą od kontrolowanych warstw, metody próbkowania, tolerancji numerycznej i modelu przeciwnika.

Stan: ZAPROJEKTOWANE na poziomie koncepcji; **NIEZAIMPLEMENTOWANE; NIEZMIERZONE** adversarialnie.

! **Granica**: to nie jest dowód w sensie SNARK-a. Nowsze analizy opisują ataki mieszczące się **poza kontrolowanym top-k** ostatniej warstwy. TopLoc ma być **sygnałem w protokole**, nie wyrokiem — stąd potrzeba losowego, wielopunktowego audytu i roli niezależnego weryfikatora.

! **To nie jest nasz wynalazek — to stan sztuki**. Metoda opublikowana (arXiv:2501.16007, styczeń 2025); Prime Intellect użył jej do weryfikacji rolloutów w INTELLECT-2 (arXiv:2505.07291 — rozproszony trening 32B na cudzym sprzęcie). **Nasza robota jest w sieci wokół tego** (rozliczenia, wybór węzła, kara, ekonomia), nie w samej metodzie.

5.2. VRF (losowanie zadań). Klient nie wybiera node'a, więc nie kupi „swojego”. Losowanie per sesja, nie per tura (uzasadnienie w §6).

5.3. Reputacja + kara. Stake w grze; fałszywy wynik = utrata stawki.

Stan faktyczny (uczciwie): dziś kod weryfikuje **tylko podpis**, nie aktywację. `activation_hash` jest zwykłym stringiem — **nikt go nie przelicza**. Node, który sam podpisze fałszywy hash i dowolny tekst, przechodzi całą weryfikację. Receipt dowodzi dziś „zarejestrowany node X tak twierdzi”, nie „policzył”.

Prawdziwy moment prawdy: Frank liczy model, Szpon weryfikuje, oszust zostaje złapany.

6. Podstuch — granica, o której trzeba mówić

Dwa różne ataki, nie wolno ich mieszać:

Atak	Czy TopLoc łapie
Node udaje model X, przekazuje do Claude/GPT	TAK — hashe aktywacji nie zgodzą się z wagami
Farma zbiera prompty do destylacji	NIE — node liczy uczciwie, tylko zapisuje

Kluczowa liczba (ESTYMACJA, niezmierzona):

Udział podsłuchującego	Losowanie per tura, 50 tur	Jeden node na sesję
1%	39,5%	1%
5%	92,3%	5%
10%	99,5%	10%

Skąd ta różnica: harnessy agentowe wysyłają **cały kontekst przy każdej turze** (pliki, historię, kod). Więc losowanie per tura nic nie daje — jedna tura już zawiera wszystko. **Trzeba losować node RAZ NA SESJĘ.**

Trzy niewygodne wnioski:

1. **Najwięcej GPU mają korporacje i państwa.** Laboratorium z 10% mocy zbiera prawdziwe zadania programistów — najcenniejsze dane treningowe. **A SIMON płaci mu za to w THINK.**
2. **Publiczna sieć MVP chroni kod GORZEJ niż komercyjne API.** API ma przynajmniej umowne zasady użycia danych; anonimowy node nie ma żadnych. Hasło „nie karm korporacji” jest w tej wersji **nieprawdziwe.**
3. **Nie da się tego naprawić kryptografią na domowym sprzęcie.** FHE ≈ 3 s/token; poufne obliczenia GPU (TEE) istnieją tylko na kartach serwerowych — czyli sprzęcie korporacji.

Co realnie działa (od najmocniejszego):

1. **Prywatne pody** — użytkownik wybiera „tylko node'y z mojej listy” (swoje, znajomych, firmy). **Jedyna prawdziwa ochrona na domowym sprzęcie.** Dla kodu powinno być DOMYŚLNE.
2. **Anonimizacja u klienta** przed wysłaniem — ograniczenie: logikę kodu dalej widać.
3. **Kanarki na każdy node** — wyciek wskazuje sprawcę i uzasadnia utratę stawki. Działa tylko przy publicznym wycieku.
4. **Jawna etykieta:** „sieć publiczna = traktuj prompt jak publiczny”.

Trop zbadany i zamknięty (żeby nikt nie wracał): „klucz nałożony na model” (szyfrowanie wag/aktywacji po stronie klienta). Sprawdzone eksperymentalnie — **nie działa:**

„Da się zaszyfrować rurę, nie da się zaszyfrować komputera, który liczy za ciebie.”

Klucz na wagach nic nie szyfruje (przestawienie neuronów nie zmienia funkcji). Klucz na aktywacjach rozbija się na ReLU i LayerNorm. Nawet gdy maska przechodzi — **model jest publiczny**, więc węzeł odzyskuje klucz dzieleniem wag. Trzeciej drogi nie ma: albo FHE/MPC, albo TEE.

7. Ekonomia

Nowością w SIMON jest weryfikacja inferencji, nie ekonomia tokena. Token economy to najlepiej udokumentowane pole minowe w krypto — błędy innych są opisane i można ich ominąć.

Dziewiąta pułapka — wash-compute (dopisana 2026-09-17 po audycie zewnętrznym):

Audyt proponował lek: „emisja $\leq$ część netto spalonych opłat”. **To nie usuwa mechanizmu — tylko przesuwą próg.** Farma, która i tak potrzebuje compute, dalej ma dodatni bilans: dostaje pracę i część emisji. Dla farmy AI wash-compute to nie oszustwo, ale **optymalizacja** — liczę u siebie zamiast u kogoś. Póg emisji ustala wielkość bonusu, nie zamyka drogi.

Bilans podmiotu kontrolującego klienta i wykonawcę: **zysk = emisja - nieodwracalna opłata - prąd - koszty weryfikacji**. Część płatności dla executora wraca do tej samej kieszeni. Jeśli emisja przekracza koszty nieodwracalne, samotransakcja pozostaje rentowna.

Na obecnym etapie nie znamy permissionless mechanizmu, który odróżnia realny popyt od ekonomicznie równoważnej samotransakcji. Dlatego emisja zależna od wolumenu pracy pozostaje ZABLOKOWANA.

Najczystszy model MVP (bez emisji per-work): klient płaci executorowi za usługę; verifier dostaje część tej samej opłaty; protokół pobiera niewielką opłatę settlementową; **nie ma dodatkowej emisji za wykonany job**. THINK może być niewymienialnym kredytem rozliczeniowym, a emisja SIMON nie jest uruchamiana przed symulacją i wykazaniem realnego, niepowiązanego popytu. **To usuwa rentowność wash-compute** — można dalej zlecać pracę samemu sobie, ale bez dopłaty protokołu nie ma czego wydobywać.

Wniosek: wash-compute jest nierozwiązywalny na poziomie protokołu. Protokół nie wie, czy zlecający i node to ta sama osoba — chyba że dane tożsamościowe są jawne, a wtedy łamie się prywatność promptu. To **czwarta granica** obok podsłuchu, alignmentu i odwracalności aktywacji — nie luka do załatania, a granica do nazwania.

Precedens:

Projekt	Co robił	Lekcja
Golem (2016)	rynek mocy za token	podaż była, popytu nie było przez lata
Helium	nagrody za hotspoty	setki tysięcy hotspotów, przychód groszowy; fałszowanie lokalizacji
Filecoin	emisja za dysk	Filecoin Plus → fikcyjne umowy z sobą
Livepeer	zweryfikowane transkodowanie + kara	technicznie najbliższy SIMON
Helium DC / Render	burn-and-mint	gotowy wzorzec na „bon na compute”

Wspólny błąd wszystkich: emisja dla dostawców rośnie szybciej niż popyt użytkowników. Sieć rośnie sprzętem, a nie użyciem — emisja finansuje farmy, nie produkt.

Proponowany wzorzec (PROPOZYCJA, nie decyzja):

SIMON – token rzadki, PRZELEWALNY
emisja: stała na epokę, dzielona wg udziału w OPŁACONEJ,
ZWERYFIKOWANEJ pracy (nie od liczby tokenów!)
⇔ portfel do portfela: OK

THINK – bon, NIEPRZELEWALNY
powstaje: SPALENIE SIMON, po stałej cenie za jednostkę compute
zużywany na zlecenia
NIE da się sprzedać → nie da się spekulować,
ani kupić pierwszeństwa kopacza

Dlaczego bon nieprzenośny: kopacz ma pierwszeństwo przed kupującym (zasada spółdzielni). Gdyby THINK dało się przelać, wieloryb kupiłby THINK od kopaczy **i razem z nim ich pierwszeństwo** – rynek obchodzi zasadę jednym przelewem.

Dlaczego emisja stała na epokę: „X THINK za N tokenów” = inflacja zależna od postępu NVIDIA. Stała emisja na epokę (jak trudność w BTC) daje szybszym kartom większy udział, ale **nie powiększa podaży**.

Dlaczego cena stała za compute: użytkownik musi móc zaplanować budżet, a kopacz płaci prąd w złotychkach. Zmienny tylko token rzadki.

Prawo i podatki – bramki, nie przypisy: MiCA (token wymienialny oferowany publicznie = prawdopodobnie obowiązek white papera; wymiana = licencja CASP), PIT w PL (zapłata kryptowalutą za usługę = prawdopodobnie każde zlecenie zdarzeniem podatkowym, co zabija UX). **Wymaga prawnika, nie zgadywania.**

Kolejność: ekonomia **po weryfikacji**. Dopóki oszust nie jest łapany, emisja nagradzała by oszustów.

8. Governance – dławienie, nie ban

Problem: „bezpieczny / niepatologiczny” nie jest mierzalne. Ktoś musi zdecydować. Centralny komitet = moloch, przed którym uciekamy. Rynek = patologia też może być popularna.

Rozstrzygnięcie: sieć reguluje STOPNIEM, nie wyrokiem.

Model	Co się dzieje	Odwracalne?
Używany, sprawny, bezpieczny	pełny przydział hostów i compute	—
Wątpliwy	mniej hostów / mniej compute	TAK
Używany do ataków/zła	degradacja do minimum (nie zero)	TAK

Cztery powody, dlaczego stopniowość:

1. **Nie ma centralnego wyroku** – jest funkcja przydziału, działa automatycznie.
2. **Odwracalne** – model, który naprawił problem, odzyskuje zasoby. Ban jest wyrokiem na zawsze; degradacja jest **stanem**, nie etykietą.

3. **Zgodne z zasadą „użycie decyduje, który model żyje”** — zły model nie jest zakazany, po prostu nikt go nie używa i nie dostaje zasobów.
4. **Zgodne z zasadą spółdzielni** — przydział jest ekonomiczny.

Luka, nazwana jawnie: skąd sieć wie, że model jest używany do zła? Klasyfikator treści **jest sprzeczny z zasadą prywatności promptu** — żeby wykryć „zło w treści”, trzeba widzieć treść. **Realizowalne tylko przez detekcję po zachowaniu (wzorce na poziomie sieci, nie treści) i zgłoszenia klientów.**

Uczciwie: mechanizm stopniowej degradacji jest **hipotezą, nie sprawdzonym rozwiązaniem**. Nie ma dowodu, że realnie odcina patologię — bo zły użytkownik może po prostu użyć innego modelu. Wymaga symulacji.

9. Co jest ZMIERZONE (nie oszacowane)

Składnik	Wartość	Źródło
TTFT	1,74 s	ZMIERZONE (vLLM, prefix caching OFF)
Generowanie 27B na RTX 3090	21,8 tok/s	ZMIERZONE
Prefill (sesje 500→36 500 tok.)	~2500-2900 tok/s, stabilny do 99% limitu	ZMIERZONE
Błąd heurystyki znakowej (proza PL)	<2%	ZMIERZONE
Błąd heurystyki (gęsty JSON)	~48%	ZMIERZONE
Błąd heurystyki (base64)	~65%	ZMIERZONE
Błąd heurystyki jako funkcja długości	stały procent, nie zależy od długości	ZMIERZONE
RAM spill powyżej progu	nie występuje (pula KV pre-alokowana przy starcie)	ZMIERZONE do 99% limitu
Sieć (transport)	~12 tok/s	! ZAŁOŻENIE, niezmierzone
Weryfikacja TopLoc	~1-3 s	! szacunek z paperu

Wniosek z pomiarów heurystyki: błąd zależy od **typu treści nieznanego z góry** (proza/JSON/base64), nie od długości. **Nie da się go załatać jednym mnożnikiem** — jedynym wyjściem jest pytanie tokenizera o prawdę (`/tokenize`). To zamknęło dyskusję „czy heurystyka wystarczy”.

Odpowiedź na 500 tokenów: ~25-30 s — i użytkownik **nic nie widzi przez cały czas**, bo wynik pokazywany jest po weryfikacji.

! **Największa rzecz do przemyślenia.** „Weryfikuj przed pokazaniem” zabija UX przy 22 tok/s. API DeepSeeka oddaje to samo w 2-3 s i strumieniu. Alternatywa: **optymistyczne zdanie** (strumieniu od razu, sprawdzaj wyrywkowo po fakcie, oszusta karz utratą stawki). **Po tych liczbach warto do tego wrócić. To jest otwarte, nie rozstrzygnięte.**

10. Granice — jawnie, bo obrona nie może milczeć

Problem	Stan
Oszust fałszujący wynik	! ZAPROJEKTOWANE (TopLoc) — kod nie weryfikuje aktywacji , więc dziś nie jest łapany
Podstęp przed node	✗ nie do naprawienia na domowym sprzęcie (FHE ~3 s/token, TEE tylko serwerowe)
Node udający proxy do Claude/ GPT	! ZAPROJEKTOWANE — TopLoc to łapie pod warunkiem działającej weryfikacji
Farma destylacyjna (zbiera prompty)	✗ nie do wykrycia — liczy uczciwie
Odwracalność aktywacji	✗ prompt da się odtworzyć z aktywacji (arXiv:2505.18332) — „node widzi aktywację, nie tekst” nie jest gwarancją prywatności
Alignment semantyczny (czy model zatruty)	✗ OTWARTE — świadomie nierozwiązane
Sybil (jeden operator = 1000 nodów)	! wymaga symulacji
Prywatność promptu w publicznej sieci	✗ 1 node widzi tekst (decyzja MVP)

Wniosek dla użytkownika: jeśli prompt jest cenny, używaj **prywatnych podów**. Publiczna sieć MVP chroni kod **gorzej niż komercyjne API**.

11. Podaż i popyt

Wąskie gardło to POPYT, nie podaź. Ludzie chętnie kopią (BOINC, Salad), ale dlaczego ktoś ma **kupić** THINK, skoro API DeepSeeka ≈ \$0,60/mln tokenów i odpowiada 10× szybciej?

Realne źródła popytu:

1. **Kopacze zużywają własne tokeny** — jedyny naturalny popyt od dnia 1

2. Odporność na cenzurę / brak jednego dostawcy
3. Prywatność — **ale decyzja MVP (jeden node widzi prompt) ją podważa**. Nie da się tego sprzedawać jako „prywatne”.

Baza modeli — pamięć karty wyznacza podaż (Q4 ≈ 0,6 GB / mld parametrów):

Karta	Model realnie
8 GB	7-8B
12 GB	≤14B
24 GB	27-32B dense / MoE ~30B
48-96 GB	70B, większe MoE
~475 GiB (DeepSeek V4.1)	poza zasięgiem — wymaga lokalnych podów, nie WAN

Konsekwencja: w pierwszym roku SIMON sprzedaje otwarte modele do ~30B, **nie czołówkę**. Akceptowalne dla kodu, streszczeń, przetwarzania wsadowego. **Nie konkuruje z najlepszymi modelami agentowymi**.

Etapy:

Etap	Aktywne node'y	Założenie
Alfa zamknięta	2-10	własne maszyny + znajomi
Publiczna alfa	instalacji setki-tysiące, aktywnych po 30 dniach 50-300	typowy spadek po premierze
Token na rynku	tysiące szybko, ale farmy i spekulanci	sybil + prawo (MiCA, KYC/AML) to bramki

12. Stan implementacji

Kod: ~5800 linii Rusta (+ testy). Rdzeń protokołu, transport (libp2p), koordynator, node, harness.

Zrobione i zweryfikowane na żywo (2026-09-17):

Element	Dowód
Bramka limitu kontekstu oparta na prawdzie (/ tokenize)	smoke: deklaracja 1000 vs realnie 1400 → kontrolowana odmowa
Chunkowanie + map-reduce dla plików > limit node'a	live: 20/20 kawałków, redukcja 20→2→1, zero timeoutów
łańcuch agent→agent (CODER→TESTER)	mock + live, obie fazy z weryfikacją receiptu
peer_id per-proces (był identyczny dla wszystkich nodów)	live: dwa node'y → różne ID
Dwa równoczesne agenty na dwóch GPU	live: Szpon/qwen3.8-27b + Franko/Bielik
Transport: domyślny timeout 10 s w libp2p (ukryty bug)	znaleziony na żywo, podniesiony do 300 s
Przenośność (macOS)	audyt: zero zależności z natywnym C; fix temp_dir()

Czego brakuje:

Element	Status
Weryfikacja aktywacji (TopLoc)	ZAPROJEKTOWANE — to jest moment prawdy
Księga z ostatecznością transakcji	BurnProof dziś syntetyczny
Warstwa ACP	zaprojektowana, nie zaimplementowana
Symulacje (sybil, ekonomia)	nieuruchomione
Router modeli (sleep mode)	plan, nie kod

Uczciwie: 50-80% dzisiejszego kodu zostanie przepisane przed produkcją. To normalne i jest powodem, żeby format zamrozić dopiero po weryfikacji.

13. Roadmapa

M0 (dziury w protokole) → **M1** (transport + binarka) → **M2** (dwa node'y, prawdziwy vLLM) → **M2.5** (TopLoc + oszust złapany) → **M2.7** (zamrożenie formatu + recenzja) → **M2.9** (10-12 przypadków) → **M3** (warstwa ACP).

M2.5 to moment prawdy. Wszystko przed nim to przygotowanie, wszystko po nim to rozbudowa.
Dopóki oszust nie zostanie złapany — nie wiemy, czy SIMON działa.

Czego świadomie NIE robimy: federated learning (nikt nie potrzebuje do MVP), guardy per specjalizacja (najpierw pomiar), „kontekst przed modelem” (test kontrolny pokazał obejście, nie naprawę), rejestr koordynatorów jako governance (statyczny plik = świadome uproszczenie MVP).

14. Zasady, które wyszły z tej pracy

1. Bez dowodu nie ma statusu. Trzy statusy zamiast „zrobione”: - `ZAPROJEKTOWANE` — decyzja istnieje - `ZAIMPLEMENTOWANE` — tylko ze wskazaniem testu, który **padał przed poprawką** - `ZMIERZONE` — tylko ze wskazaniem pliku z wynikiem

Powód: cztery razy w jednej sesji status był zapisany szybciej, niż został sprawdzony. Roadmapa twierdziła „Computational Integrity ZROBIONE” — kod tego nie realizował.

2. Recenzja przed commitem formatu, nie po. Recenzent znalazł 4 dziury w kodzie z 50 zielonymi testami, potem jeszcze 3 (podpisany `OrderAccepted`, `order_id` z licznika, brak weryfikacji obliczeń).

3. Uczciwość o granicach jest częścią produktu. Dokument, który milczy o tym, czego nie umie, jest gorszy niż brak dokumentu — bo ludzie podejmują na jego podstawie decyzje.

15. Otwarte pytania

1. Czy pokazywać wynik przed weryfikacją? (UX vs weryfikacja — nierozstrzygnięte)
 2. Czy SIMON publicznie obiecuje „twoje dane nie idą do korporacji”? Uczciwie da się to obiecać tylko w prywatnych podach.
 3. Skąd sygnał „model używany do zła”? (zachowanie / zgłoszenia — nigdy treść)
 4. Jak mierzyć degradację modelu? (utrata hostów / priorytet kolejki / mnożnik stawki)
 5. Czy stopniowa degradacja realnie odcina patologię, czy tylko przenosi ją na inny model? (wymaga symulacji)
 6. Jaki jest stosunek wybite:spalone w emisji? (wymaga symulacji)
 7. Czy sieć publiczna ma sens produktowo, skoro chroni gorzej niż API? (prywatne pody jako odpowiedź)
-

16. Jednym zdaniem

SIMON to miejsce, gdzie otwarte wagi zamieszkują — ludzie z GPU liczą, sieć sprawdza, że policzyli naprawdę, a model żyje tak długo, jak długo jest używany, sprawny i bezpieczny.

Dokumentacja decyzji: `memory/decisions/` (9 plików architektonicznych + 70+ pozycji D-*). Roadmapa: `ROADMAP.md`. Kod: `crates/` (Rust, ~5800 linii). Zasada nadrzędna: bez dowodu nie ma statusu.